INFORMATIONSBLATT

IB02 – LEFT JOIN und Auswertungen über mehrere Tabellen

SQL · LS07 – Tabellen verbinden (JOIN) · IB02 | 2BKI2 · Schuljahr 2026/2027

🎯 LERNZIEL

Nach diesem Informationsblatt können Sie …

- den Unterschied zwischen INNER JOIN und LEFT JOIN erklären,
- mit LEFT JOIN … IS NULL Datensätze ohne Partner finden,
- JOIN mit GROUP BY und Aggregatfunktionen kombinieren (z. B. Umsatz pro Kunde).

1 · INNER JOIN verliert Zeilen

Ein INNER JOIN zeigt nur Zeilen, die in **beiden** Tabellen einen Partner haben. Das Tandem hat kein Fahrrad – es fehlt:

```
SELECT t.Bezeichnung, f.FahrradNr
FROM fahrradtyp AS t
  INNER JOIN fahrrad AS f ON t.TypNr = f.TypNr
WHERE t.Bezeichnung IN ('Lastenrad', 'Tandem');
```

▶ ERGEBNIS

Bezeichnung	FahrradNr
Lastenrad	12
Lastenrad	16

2 Zeilen

2 · LEFT JOIN behält alle Zeilen der linken Tabelle

Beim `LEFT JOIN` kommt **jede** Zeile der linken Tabelle (der vor dem JOIN) ins Ergebnis. Fehlt der Partner, füllt die Datenbank die rechten Spalten mit NULL:

```
SELECT t.Bezeichnung, f.FahrradNr
FROM fahrradtyp AS t
  LEFT JOIN fahrrad AS f ON t.TypNr = f.TypNr
WHERE t.Bezeichnung IN ('Lastenrad', 'Tandem');
```

▶ ERGEBNIS

Bezeichnung	FahrradNr
Lastenrad	12
Lastenrad	16
Tandem	*NULL*

3 Zeilen

Das ist der Schlüssel zu einer häufigen Frage: **„Wer hat keinen …?“** – LEFT JOIN und dann die Zeilen mit NULL herausfiltern:

```
SELECT k.KundenNr, k.Nachname, k.Vorname
FROM kunde AS k
  LEFT JOIN ausleihe AS a ON k.KundenNr = a.KundenNr
WHERE a.AusleihNr IS NULL;
```

▶ ERGEBNIS

KundenNr	Nachname	Vorname
14	Klein	Noah
15	Wagner	Lina

2 Zeilen

> ✅ **MUSTER „OHNE PARTNER“**
>
> `FROM A LEFT JOIN B ON … WHERE B.primärschlüssel IS NULL` liefert alle Zeilen aus A, zu denen es in B nichts gibt.

> 💡 **RIGHT JOIN**
>
> Es gibt auch `RIGHT JOIN` (behält alle Zeilen der rechten Tabelle). Man braucht ihn selten – man kann einfach die Tabellen vertauschen und LEFT JOIN nehmen.

3 · JOIN + GROUP BY

Jetzt lassen sich die Auswertungen aus LS06 mit Namen schreiben. Ausleihen pro Kunde:

```
SELECT k.Vorname, k.Nachname, COUNT(a.AusleihNr) AS Ausleihen
FROM kunde AS k
  LEFT JOIN ausleihe AS a ON k.KundenNr = a.KundenNr
GROUP BY k.KundenNr, k.Vorname, k.Nachname
ORDER BY Ausleihen DESC;
```

▶ ERGEBNIS

Vorname	Nachname	Ausleihen
Anna	Meier	4
Tim	Braun	4
Jonas	Wolf	3
Paul	Weber	2
Marie	Keller	2
Felix	Koch	2
Sophie	Becker	2
Lukas	Schmid	2
… 7 weitere Zeilen …		

15 Zeilen

`COUNT(a.AusleihNr)` statt `COUNT(*)`: So bekommen Kunden ohne Ausleihe eine 0 (NULL wird nicht gezählt). Mit COUNT(*) stünde dort eine 1.

Und die wichtigste Zahl für Frau Berger – der **Umsatz**. Pro Fahrrad einer Ausleihe kostet es Tage × Tagespreis:

```
SELECT k.Nachname, SUM(a.Tage * t.Tagespreis) AS Umsatz
FROM kunde AS k
  INNER JOIN ausleihe      AS a  ON k.KundenNr   = a.KundenNr
  INNER JOIN ausleihposten AS ap ON a.AusleihNr  = ap.AusleihNr
  INNER JOIN fahrrad       AS f  ON ap.FahrradNr = f.FahrradNr
  INNER JOIN fahrradtyp    AS t  ON f.TypNr      = t.TypNr
GROUP BY k.KundenNr, k.Nachname
ORDER BY Umsatz DESC
LIMIT 5;
```

▶ ERGEBNIS

Nachname	Umsatz
Meier	459.00
Schäfer	406.00
Wolf	304.00
Fischer	250.00
Keller	215.00

5 Zeilen

> 💡 **GRUPPIEREN NACH DEM SCHLÜSSEL**
>
> Man gruppiert nach `k.KundenNr` (und nimmt den Namen dazu) – zwei verschiedene Kunden könnten gleich heißen.